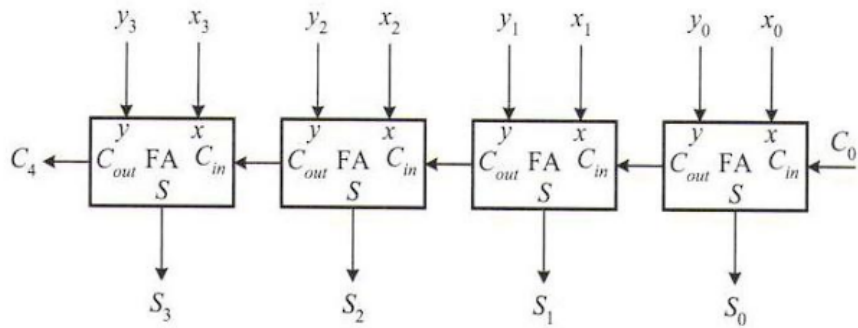


Digital Logic Design – Lab 2

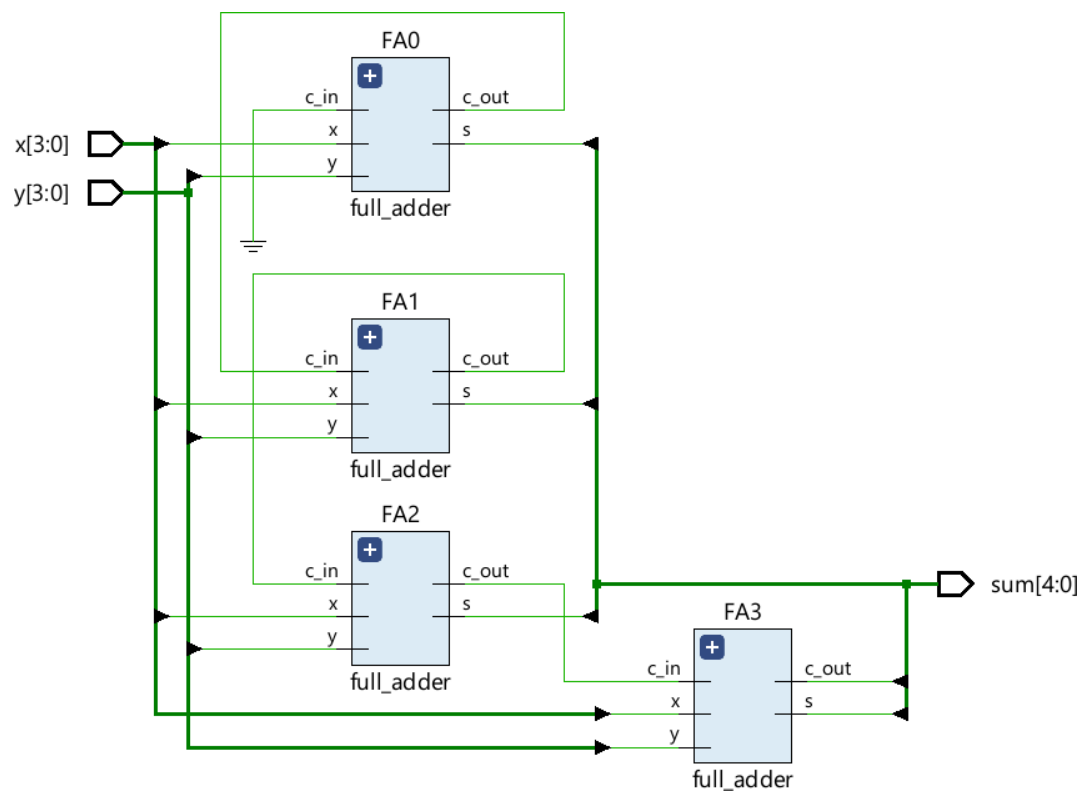
1. Topic: 4 Bits Adder, and 4x4 Multiplier

2. Design structure:

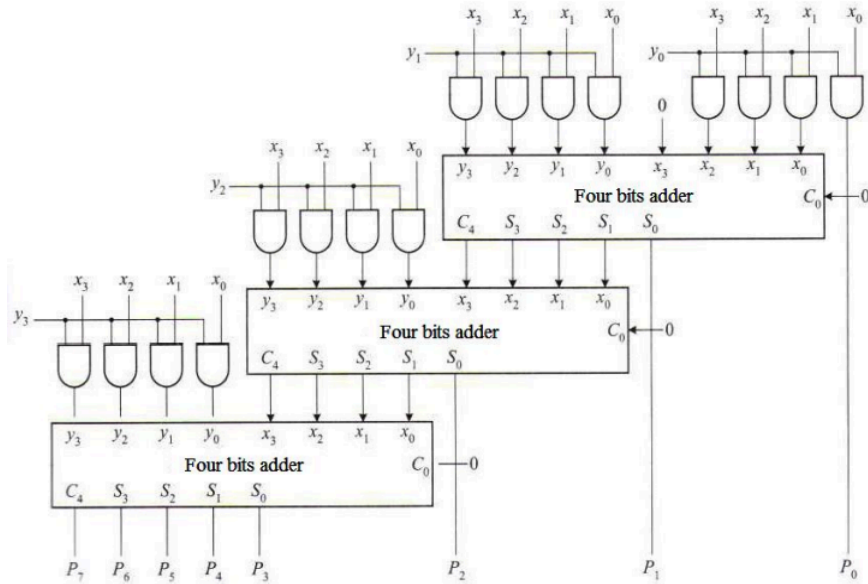
- 4 Bits Adder



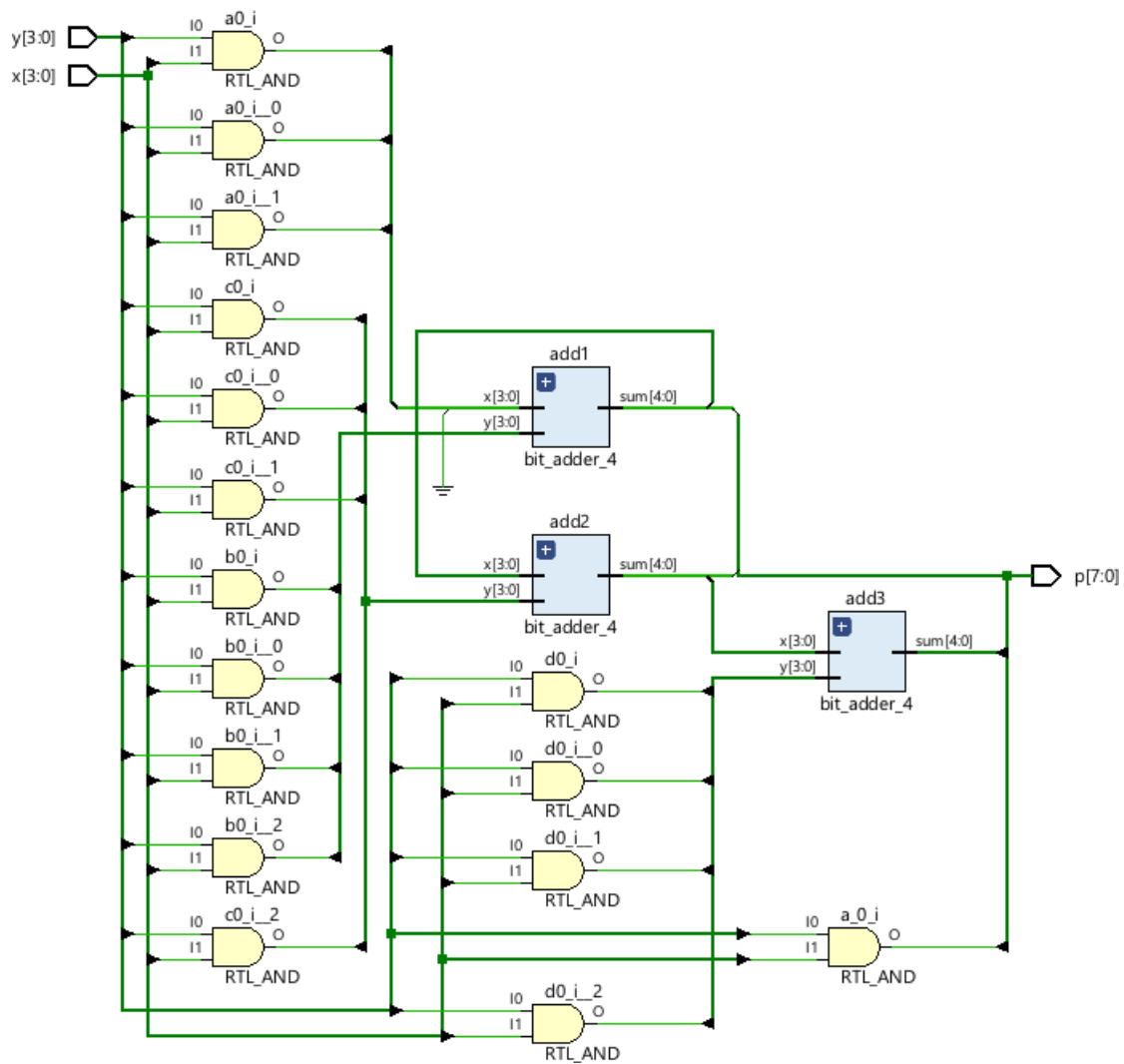
Also, this is the schematic generated by Vivado



- **4x4 Multiplier**



Also, this is the schematic generated by Vivado



3. Code and Test bench: the codes are zipped in same .zip as this report

- 4 Bits Adder

```
22 // 4 Bits Adder implemented by using the full_adder module
23 module bit_adder_4(x, y, sum);
24     // input ports
25     input [3:0] x, y;
26
27     // wire for intermediate variables
28     wire [2:0] c;
29     wire c4;
30     wire [3:0] s;
31
32     // output ports
33     output [4:0] sum;
34
35     // Using 4 full_adder modules
36     // NOTE = full_adder(x, y, c_in, s, c_out);
37     full_adder FA0 (x[0], y[0], 1'b0, s[0], c[0]);
38     full_adder FA1 (x[1], y[1], c[0], s[1], c[1]);
39     full_adder FA2 (x[2], y[2], c[1], s[2], c[2]);
40     full_adder FA3 (x[3], y[3], c[2], s[3], c4);
41
42     // Final output of the sum
43     assign sum = {c4, s[3:0]};
44 endmodule
45
46 // Full Adder implementation using Data Flow modeling
47 module full_adder(x, y, c_in, s, c_out);
48     // input ports
49     input x, y, c_in;
50
51     // output ports
52     output s, c_out;
53
54     // Data Flow implementation
55     assign s = x ^ y ^ c_in;
56     assign c_out = (x & y) | (x & c_in) | (y & c_in);
57
58 endmodule
```

```

22 | // module bit_adder_4(x, y, sum);
23 | //     input [3:0] x, y;
24 | //     output [4:0] sum;
25 | module bit_adder_4_sim();
26 |     // input & output
27 |     reg [3:0] x, y;
28 |     wire [4:0] sum;
29 |
30 |     // Unit Under Test
31 |     bit_adder_4 uut (.x(x), .y(y), .sum(sum));
32 |
33 |     integer i, j;
34 |     initial begin
35 |         for (i=0; i<16; i=i+1) begin
36 |             x = i;
37 |
38 |             for (j=0; j<16; j=j+1) begin
39 |                 y = j;
40 |                 #100;
41 |             end
42 |         end
43 |     end
44 | endmodule

```

● 4x4 Multiplier

```

22 | // 4X4 Multiplier implemented by using the bit_adder_4 module
23 | //
24 | // module bit_adder_4(x, y, sum);
25 | //     input [3:0] x, y;
26 | //     output [4:0] sum;
27 | //
28 | module multiplier_4x4(x, y, p);
29 |     // input ports
30 |     input [3:0] x, y;
31 |
32 |     // intermediate wires
33 |     wire [3:0] a, b, c, d; // for AND gate output
34 |     wire [4:0] s1, s2, s3; // for the 4-Bit Adder output
35 |
36 |     // output ports
37 |     output [7:0] p;
38 |
39 |
40 |     // 1st. group of AND gates
41 |     assign a[0] = y[0] & x[0]; // this is p[0]
42 |     assign a[1] = y[0] & x[1];
43 |     assign a[2] = y[0] & x[2];
44 |     assign a[3] = y[0] & x[3];

```

```

46 // 2nd. group of AND gates
47 assign b[0] = y[1] & x[0];
48 assign b[1] = y[1] & x[1];
49 assign b[2] = y[1] & x[2];
50 assign b[3] = y[1] & x[3];
51
52 // 1st. 4-Bit Adder
53 bit_adder_4 add1 ({1'b0, a[3:1]}, b, s1);
54
55 // 3rd. group of AND gates
56 assign c[0] = y[2] & x[0];
57 assign c[1] = y[2] & x[1];
58 assign c[2] = y[2] & x[2];
59 assign c[3] = y[2] & x[3];
60
61 // 2nd. 4-Bit Adder
62 bit_adder_4 add2 (s1[4:1], c, s2);
63
64 // 4th. group of AND gates
65 assign d[0] = y[3] & x[0];
66 assign d[1] = y[3] & x[1];
67 assign d[2] = y[3] & x[2];
68 assign d[3] = y[3] & x[3];
69
70 // 3rd. 4-Bit Adder
71 bit_adder_4 add3 (s2[4:1], d, s3);
72
73
74 // Final output of the product
75 assign p = {s3, s2[0], s1[0], a[0]};
76
77 endmodule

```

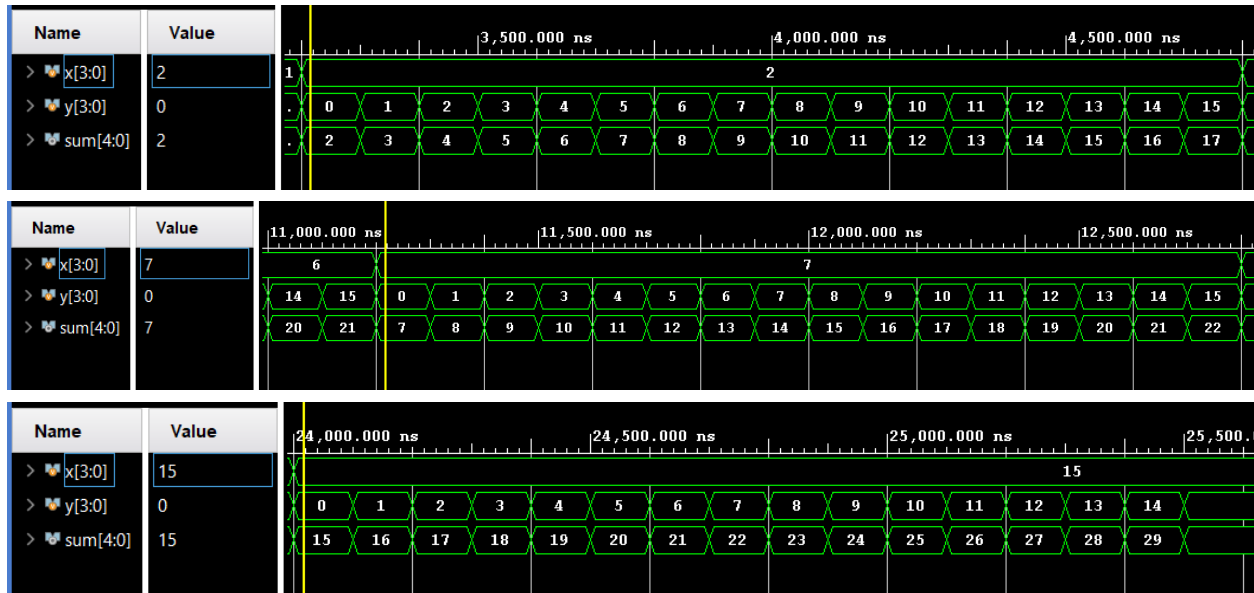
```

22 // module multiplier_4x4(x, y, p);
23 //   input [3:0] x, y;
24 //   output [7:0] p;
25 module multiplier_4x4_sim();
26 // input & output
27 reg [3:0] x, y;
28 wire [7:0] p;
29
30 // Unit Under Test
31 multiplier_4x4 uut (.x(x), .y(y), .p(p));
32
33 integer i, j;
34 initial begin
35     for (i=0; i<16; i=i+1) begin
36         x = i;
37
38         for (j=0; j<16; j=j+1) begin
39             y = j;
40             #100;
41         end
42     end
43 end
44 endmodule

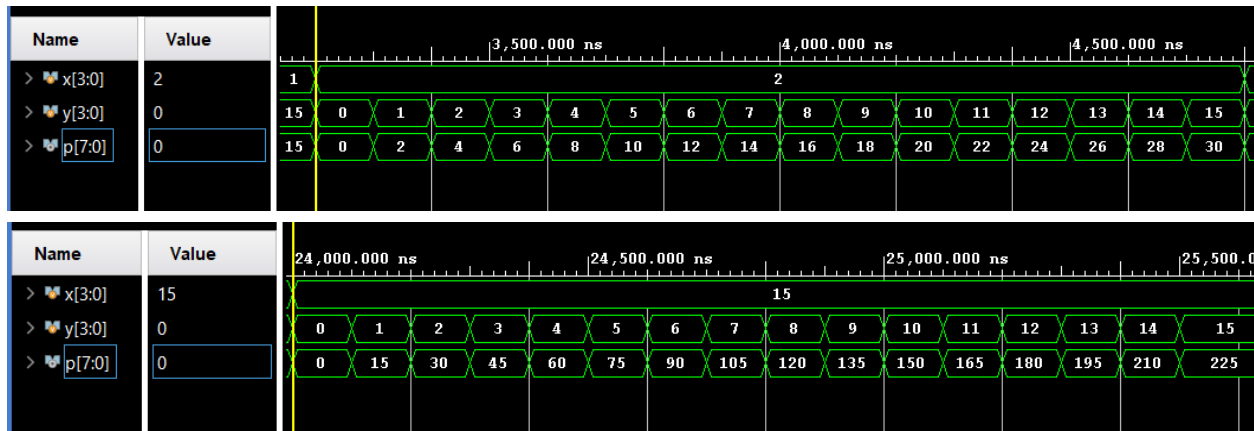
```

4. Result (Simulation waves):

- **4 Bits Adder** – some screenshots of the wave simulation, for readability purpose, the carry out of the last full adder has been concatenated with the sum of all full adders. Here we can see various examples of 4 bits sums, such as: $x = 2$ (0010) and $y = 10$ (1010) gives sum = 12 (1100), $x = 7$ (0111) and $y = 7$ (0111) gives sum = 14 (1110), and so on



- **4x4 Multiplier** – some examples of behavioral simulation. This shows the working of 4 bits by 4 bits multiplication, for easier understanding, in the simulation the numbers are being displayed as decimals instead of binary. For example, $x = 2$ and $y = 8$ gives $p = 16$, $x = 15$ and $y = 4$ gives $p = 60$, and so on



5. Reflection

In this lab, we created a 4 bits adder by making use of last class full adder module, and in the same way, created a 4x4 multiplier by using the 4 bits adder. This week we learned about using different modules to build more complex circuits

